

AEM and AECP

Introduction

The ATDECC Entity Model, is the modelled representation of what a network system's abilities. This is meant to be static for every system as an initial state. However, it provides a way to configure a system, allowing some flexibility, and permit inter-operabilities between systems so that they can understand each other.

In addition, to the AEM, the AECP, [ATDECC Enumeration and Control Protocol](#), allows every ATDECC compliant devices to communicate their entity model to each other. AECP is not only transferring AEM descriptors, but also provides a controller to take action on the system via its entity model's descriptor.

Scope

The objective is to list what would be necessary to have to comply with the [initial specifications](#). Here, the FPGA and some software control will be discussed.

ATDECC Enumeration and Control Protocol specification

The system needs to implement the necessary state machine as described in the [standard dissection](#) section.

State machines to implement:

The following state machines shall be implemented:

- The AECP [state machine](#) for the ATDECC commands
- The AECP [Vendor Unique Command](#) for Milan:
 - Protocol ID: 00-1B-C5-0A-C1-00

- GET_MILAN_INFO
- GET/SET_SYSTEM_UNIQUE_ID
- GET/SET_MEDIA_CLOCK_REFERENCE_INFO

Entity model technical specs

An entity model is a challenging thing to create inside logic. For the sake of simplicity, the entity model will be minimalistic to cover a talker and listener, starting with the latter.

Entity

Static information

- **Because of AVB**, only one entity
- The entity ID is unique, and shall be derived from the MAC address
- 3 Stored configurations, only one activate at the time:
 - Raki - 48 kHz
 - Raki - 96 kHz
 - Raki - 192 kHz

Dynamic information

The dynamic information, modeled as variables (metadata), necessary for the entity would be:

Command Name	Variable	Attributes of the variable
SET-GET_CONFIGURATION	A pointer to the current configuration index. The configuration ID change, 3 configurations as per the Entity Configuration Section	• NONVOLATILE • RESETTABLE
DE/REGISTER_UNSOLICITED_NOTIFICATION	Table of 16 controllers ID 64 bit long	
LOCK_ENTITY	A boolean providing the information whether it is linked or not	• TIMEOUTS

Entity configuration

Three configurations may be supported, for the following sample rate: 192 kHz, 96 kHz and 48 kHz.

Static information

Descriptor type	Counts	Notes
STREAM_INPUT	2 Streams input	• 1 AAF Stream • 1 CRF Stream
STREAM_OUTPUT	2 Stream outputs	• 1 AAF Stream • 1 CRF Stream
AVB_INTERFACE	1 network interface	
CLOCK_DOMAIN	1 clock domain	
CLOCK_SOURCES	3 clock sources	• 1 Internal • 1 AAF Stream derived • 1 CRF Stream derived
AUDIO_UNIT	1 audio unit for the input and output streams and input and output clusters available	
STREAM_PORT_INPUT	1 Stream output port	
AUDIO_CLUSTER	16 Audio clusters: • 8 input audio clusters • 8 output audio clusters	
AUDIO_MAP	None, everything can be dynamic	
CONTROL	1	Identity control.

Dynamic information

TODO

Non-Volatile information

The following information should be saved, and resettable upon device factory-reset.

FPGA

The FPGA implementation needs to provide flexibility for development and for the end-user. As a consequence, this raises multiple questions:

1. How to make the FPGA's entity descriptor flexible --> Yes up to a certain size
2. How to make the getter/setter generics to avoid code duplication
3. How to manage static, semi-static and dynamic info differently within the same descriptor
4. Ensure a Factory reset and Non-Volatile updates.

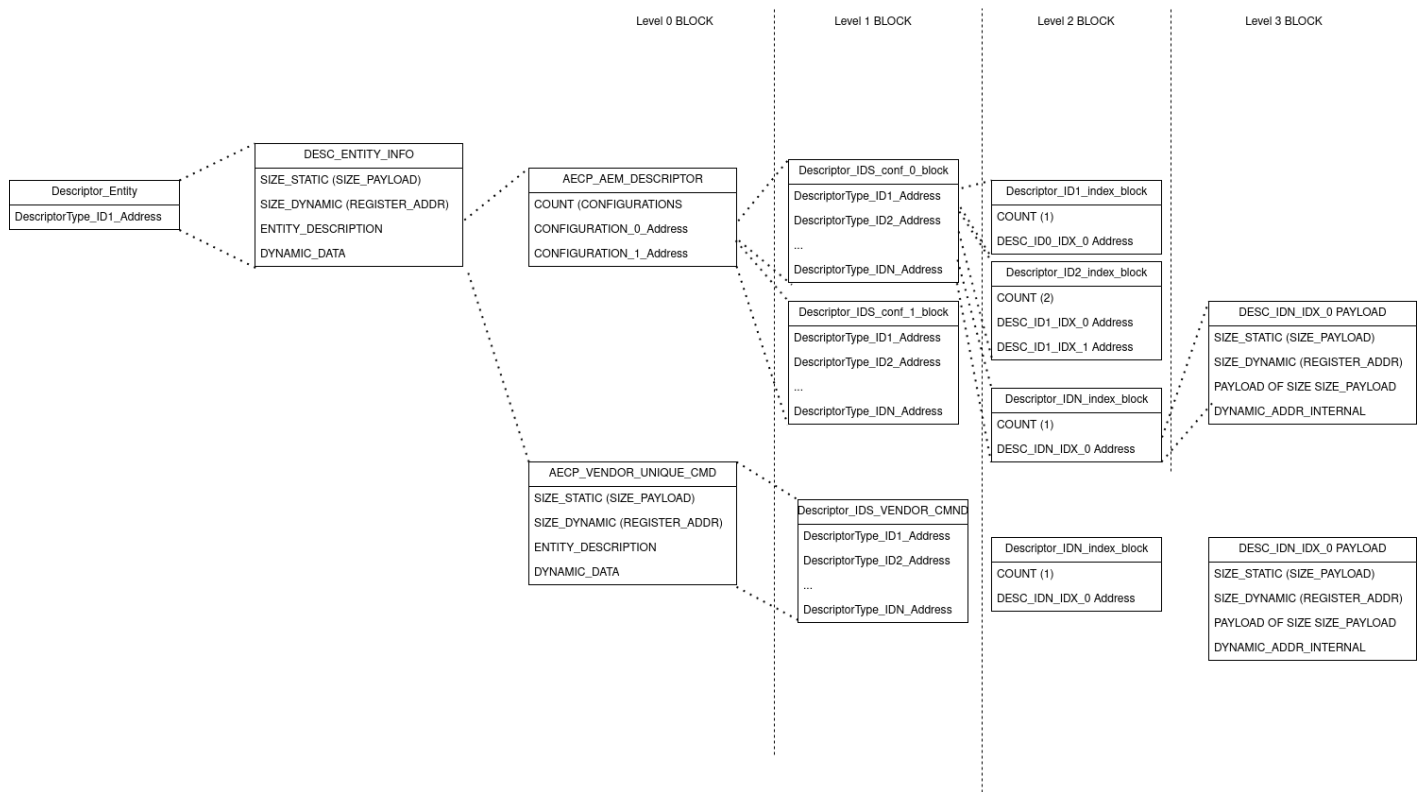
Solution

In order to, cope with the aforementioned issues, the following solutions would be necessary:

- A 4 level multi-level Block descriptions
- A generic Getter/Setter select to allow read/write of the necessary payload.
- An alias memory mapped for dynamically parameters, static, and semi-static
- A read-only, Factory defined Non-volatile memory
- A Modifiable a read/write Non-volatile memory overlaying the factory.
- A Volatile read/write for fast access, mirroring the modifiable read/write overlay.

The 4-Level block description

The Diagram below describes the 4-level block description, similar to a file system, it will handle memory-based Entity Model descriptors. There is **only one 4-level memory block descriptor per** entity.



Level	Description
Level 0	This is the ENTITY that holds information necessary to be parsed.
Level 1	The level 0 BLOCK contains a list of address ordered by descriptor ID starting from the CONFIGURATION descriptor . It lives in the RAM / BLOCK_RAM.
Level 2	The level 1 contains multiple tables. Each table is made the same with the first position the number of descriptor (count), referring to the Descriptor INDEX . Other position in the table are addresses where are located the payload of the descriptor. It lives in the RAM / BLOCK_RAM.
Level 3	The deepest level contains the size of the payload. The first place in the array contains the size of the "static, semi-static" data" The second place contains the size of the dynamic array. The Payload corresponds to the 1722.1 descriptor information held in the memory. The dynamic Address is mapped to a Multiplexed registered data. Everything lives in the RAM / BLOCK_RAM. Except where the dynamic data is pointing to.

Addresses are 16 bits, in the case when bigger than 16 bits addresses are used, they are done modulo the size of the bigger size, i.e:

- In the 32 bits are used, use an offset, and work in this offset aligned on a 16 bits boundaries.

A generic Getter/Setter

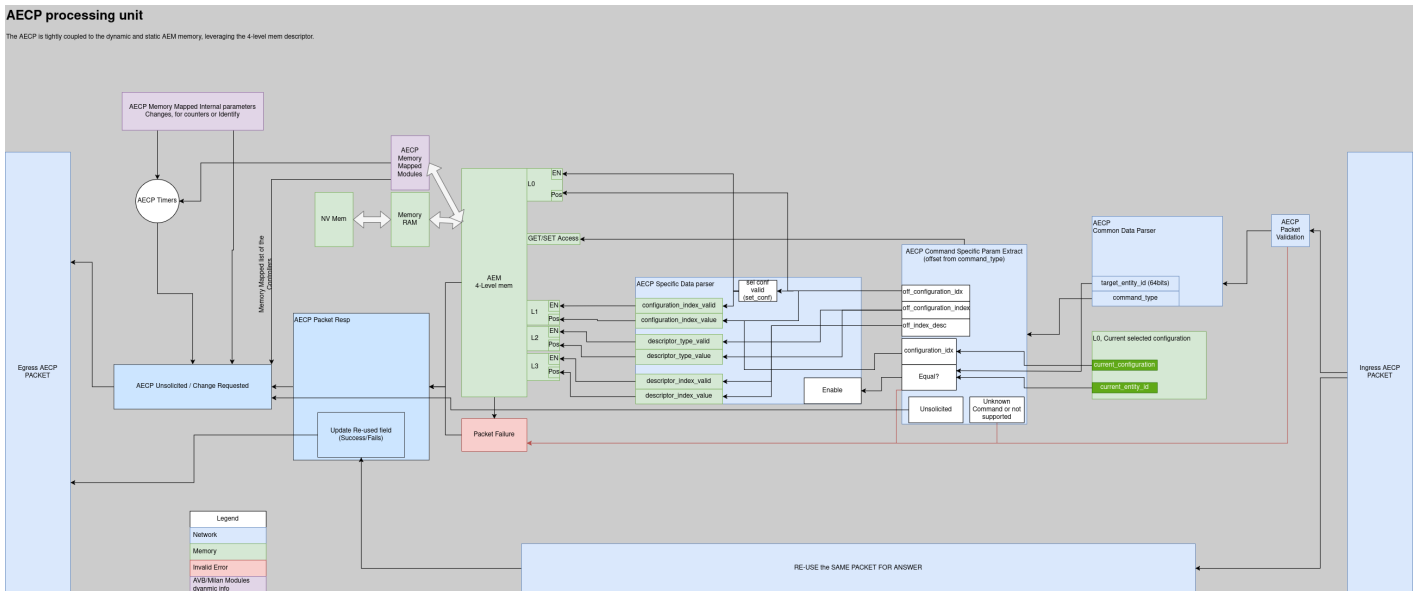
The generic Getter/Setter would allow for parsing and generate an accessor to access a specific descriptor in the memory. Based on the information from a getter setter as described in the [ATDECC Entity Model section](#), there is some information to gather to access the 3-level block memory:

Information Accessor from	Part of	Level Accessed information	Command/Response access
command_type	AECP Common Data	For command that only have this System information. Level 0 Access	• ACQUIRE_ENTITY(ONLY this parameter) • LOCK_ENTITY(ONLY this parameter) • ENTITY_AVAILABLE (ONLY this parameter) • The rest of the Commands uses it.
configuration_index	AECP Message-Specific Data	Level 1 Access	• READ_DESCRIPTOR • WRITE_DESCRIPTOR • GET_CONFIGURATION • SET_CONFIGURATION • GET_NAME • SET_NAME • GET_ASSOCIATION_ID • SET_ASSOCIATION_ID

Information Accessor from	Part of	Level Accessed information	Command/Response access
descriptor_type	AECP Message-Specific Data	Level 2 access	All Except: • GET_CONFIGUR ATION • SET_CONFIGURA TION • GET_ASSOCIATI ON_ID • SET_ASSOCIATI ON_ID • REGISTER_UNSO L.. • DEREGISTER_UN SOL.. • GET_AS_PATH • AUTH_XXX • ENABLE_TRANSP ORT_SECURITY • GET_DYNAMIC_I NFO •
descriptor_index	AECP Message-Specific Data	Level 3 access	All Except: • GET_CONFIGUR ATION • SET_CONFIGURA TION • GET_ASSOCIATI ON_ID • SET_ASSOCIATI ON_ID • REGISTER_UNSO L.. • DEREGISTER_UN SOL.. • AUTH_XXX • ENABLE_TRANSP ORT_SECURITY •

Overall Design of the volatile part

The design below aims to describe how the internal block will handle access of the different descriptor and internal states of the AECP processing Unit.



TODO ADD the interaction with the filtering database, using the AVAILABLE messages for UC traffic. ==> Done via the AECp Memory mapped.

AECp Packet Validation

Expected behaviour

- The AECp packet validation makes sure that packet **control_data_lenght** corresponds to the packet.
- The AECp **message_type** corresponds to the **AEM_COMMAND (0)**
- The AECp's **status** is **SUCCESS (0)**
- Fails when none of the written above checks pass.
 - Error message's is the AECp's status response. And it would be **BAD_ARGUMENTS (7)**

AECp Common Data Parser

Expected behaviour

- Extracts the **target_entity_id** from the **AECp Common Data**
- Extracts the **command_type** from the **AECp Common Data**
- Checks the **command_type** validity
- Checks the **controller_id**
- Fails when:
 - The **command_type** does not exist.

AECp Command Specific Extract

Expected behaviour

- From the Command specific part, and according to the table referred in section [A generic Getter/Setter](#), pass an offset in the packet and validity signal:
 - The offset and validity of the **configuration_index** within the packet
 - The offset and validity of the **descriptor_type** within the packet
 - The offset and validity of the **descriptor_index** within the packet
- Provides to the AEM whether it is a **getter or setter**.
- Provides whether this command expects to send **Unsolicited** notification or not.
- Fails, when :
 - The **target_entity_id** is not equal to **Current Selected configuration, L0, BAD_ARGUMENTS (7) (TODO verify with the spec)**
 - The **command_type** is not supported. Error code is **NOT_IMPLEMENTED (1)**
 - The entity is locked and if the command does not perform when the entity is locked, then the command returns **ENTITY_LOCKED (3)**

AECP L0 Current Selected Configuration

Expected behaviour

- At all time, provide the information about:
 - The **current_configuration**
 - The **current_entity_id**
- If none of these are available for any reason
 - Return a message with **ENTITY_MISBEHAVING**

AECP Specific Data Parser

Expected behaviour

- If enabled:
 - Pass down the parameters Valid and Index information
 - The **configuration_index** validity will behave differently to comply with the 1722.1 Specification
 - If the packet is a **command_type** of **SET/GET_CONFIGURATION**, then it will use the packet's **configuration_index** information
 - Otherwise it will use L0 current status
- Otherwise:
 - Do not care about the values,
 - Make sure the Enable signals are set to 0.

AECP Packet response

Expected behaviour

- Buffer the entire packet
- Adds the necessary payload depending on **command_type**.
 - For the **SETTER**, the same payload is returned.

- For the **GETTER** the same packet should be returned, with the added state of the memory.
- Adds the **status** provided from the upstream modules (Any Status code, error, or success)
- Set the **message_type** set
 - AEM_RESPONSE
 - VENDOR_UNIQUE_RESPONSE if a vendor unique

AECP Unsolicited / Change Request notifications

Expected behaviour

- The system can keep up to 16 Controller_ID in the L0 **Dynamic** memory of the system
- A Signal from the [AECP Command Specific Extract](#) whether it is unsolicited or not.

AECP Timers

Expected behaviour

- The timer shall provide a timeout for:
 - The 1-minute timeout from the ENTITY_LOCK command, Unlocks after the 1-second
- Shall provide a 1-second throttle mechanism upon GET_COUNTER updates from internal counters.
- Unsolicited/Solicited, remove from the DB after not receiving AVAILABLE_CONTROLLER for some time.

AECP Memory Mapped module

TODO: Provide the AECP MEMORY MAPPED module dependent on the descriptor and the configuration of a descriptor.

The list of the different data that are directly linked to a MILAN/AVB descriptor, and as a result dependent on commands or unsolicited-notification such as **GET_COUNTERS**:

- Stream Port IN/OUT
 - The specified Milan and AVB counter value.
 - Mappings with the Audio Unit to the Stream Port via the Audio Cluster configuration
- Stream IN/OUT
 - Format
 - Asynchronous Rate Converters
- Audio Units
- Milan Network interface:
 - Milan and AVB counter value.
- Identifying
- All the module supporting the CR (change request).

- Filtering database, reacting to the **AVAILABLE_CONTROLLER** message to use the **Unicast** rather than the **Multicast**.
- Database of the Controller
- PTP information

All the modules above need to be memory mapped to a specific **DESCRIPTOR**. That's part of the **DYNAMIC** information map from the **AEM_MEMORY**.

Revision #23

Created 2025-05-17 15:11:49 UTC by Mr.M

Updated 2025-06-04 06:53:35 UTC by Mr.M